Class Imbalance Lab 1

Authors: Mariel Concepcion, Lorenzo De Villa, Bingbong Manglicmot, Caitlin Sebastian, JF Viray

Learning Team: 7

---

1. Executive Summary

CreditByte is a mid-sized financial services company, which struggles with fraud since criminals bypass traditional rule-based systems. For example, many fraudulent transactions are small in amount, making them difficult to flag with just a simple threshold check. The goal of this study is to help improve the true classification of fraudulent transactions in order to minimize expenditures. Random Forest model was used to improve fraud detection with particular attention to the heavy class imbalance between the 305 fraudulent and 180,149 non-fraudulent samples.

Among the various methods tested, Random Undersampling achieved the highest recall of 93%, but this came at the cost of terrible precision (53%), leading to many non-fraudulent transactions being incorrectly flagged. By contrast, SMOTE delivered a lower recall of 91% but significantly improved precision to 94%, representing a more balanced trade-off between recall and precision. In terms of monetary savings, SMOTE is a more practical and business-friendly choice since you get to save 3,341; unlike random undersampling that has a loss of 238,896. The team further built on SMOTE by combining it with other strategies but only improved marginally. Overall, these findings highlight machine learning as a stronger defense against fraud for CreditByte than just thresholding. Moving forward, future studies should expand on these results by exploring alternative models (e.g., gradient boosting or deep learning), and testing ensemble approaches and real-time detection systems to ensure CreditByte remains adaptive to evolving fraud tactics. Additionally, research could incorporate explainability tools like SHAP or LIME to ensure proper interpretability for stakeholders.

(The project environment is set up with Python version 3.12.8.)

---

2. Introduction

In the financial industry, credit card fraud has become a pressing issue, with consumers losing $12.5 billion in 2024 alone (Federal Trade Commission, 2025). To counter this, financial services have been utilizing machine learning to detect fraud, thereby saving both consumers and providers from losses. However, CreditByte, a financial services company, has continued to rely on traditional rule-based systems. These methods, such as flagging questionably large transactions, are no longer reliable. Such strategies make these threshold-based methods ineffective and emphasizes the need for more intelligent and data-driven approaches.

This study is guided by two main objectives. First being to understand why older methods of thresholding and flagging unusual transaction amounts are ineffective. In doing so, exploratory data analysis (EDA) will be used to examine patterns that differentiate fraudulent from non-fraudulent transactions. The other objective is to improve the prediction of fraudulent transactions. This can be done by evaluating different rebalancing strategies (oversampling, undersampling, hybrid approaches, class weights, etc.), while maintaining a Random Forest classifier with 500 estimators fixed as a controlled baseline.

In applying the various methods to handle class imbalance for CreditByte's fraud detection, this study strives to build a model that both improves technical metrics such as precision, recall, and F1-score, as well as contribute to better business decisions to strengthen outcomes like detection rates. Ultimately, this study aims to support that addressing imbalance directly can provide CreditByte stronger and more efficient methods against adaptive fraud strategies.

---

3. Methodology

3.1 Data Collection

Historical transaction records on fraud cases were taken from CreditByte. The dataset included anonymized PCA-transformed features (V1-V28), transaction amounts, and outcomes (fraud or non-fraud). Its dataset contains 185,124 rows by 31 columns.

1
Collect Historical Data from CreditByte
2
Clean the Data from the Historical Dataset
3
Visualize and Analyze the Cleaned Data
4
Identify and Handle Class Imbalances
5
Build a Model and Train it using the Data

Data Dictionary:

Feature	Description	Data Type
tid	Unique transaction ID	Integer
V1 - V28	Principal Components resulting from masking sensitive features	Float
Amount	Transaction amount in USD	Float
outcome	Target Variable (Binary) 0 = Non-fraud 1 = Fraud	Integer

3.2 Data Cleaning

The data was scanned for missing values and duplicates. No missing data were found, but duplicate entries were identified and removed. A total of 4,670 duplicates were dropped, leaving 97.4% of the data for analysis.

3.3 Exploratory Data Analysis (EDA)

After cleaning the data, exploratory data analysis (EDA) was conducted to understand why the older methods of thresholding and flagging unusual transaction amounts are no longer effective.

3.4 Handling Class Imbalance

After analyzing the results of the EDA, several resampling strategies were tested to mitigate the problem of class imbalance. Standard methods including random undersampling, random oversampling, SMOTE, etc., were applied first. To maximize these methods, additional hybrid approaches were integrated with the best standard method found. These approaches include:

1. **KMeans-based resampling:** Clustering minority instances prior to oversampling for more diversification
2. **Tomek links:** Removing questionable majority samples for clearer class boundaries
3. **Hybrid Class weighting:** Combines oversampling with penalty-based learning by assigning higher weights to minority class errors
4. **Proportional sampling:** Balancing the ratio between majority and minority transactions

Apart from the standard business and technical evaluation metrics (recall, precision, F1-score, detection rate, fraud capture rate, etc.), this study introduced a new business metric called net value. Such implementation allows quantification of the financial implications of classification results. Moreover, this metric was designed to directly address Jerome's concern regarding monetary benefit of deploying fraud detection models.

3.5 Model Integration & Evaluation

After deciding the best imbalance-handling strategy, it was then integrated into the custom `FraudDetector` class within `model.py`. The Random Forest classifier of 500 estimators remained constant as the baseline and was evaluated using k-fold cross validation. This Pipeline was validated using a hold-out set from the `historical.csv` dataset. Once its performance was established, the final model was finalised using joblib, allowing for generated predictions on the `test.csv` dataset. Overall, with the dual framework (use of both business and technical evaluations), it ensures the model provides both strong statistical performance and a tangible business impact for CreditByte.

---

4. Results and Discussion

4.1 Exploratory Data Analysis

The first problem with the dataset is the relatively little data available on fraudulent transactions, with at most 305 cases recorded in the `historical.csv` dataset while having 180,149 non-fraudulent cases. As shown in Figure 1, this results in a pronounced class imbalance between fraudulent and non-fraudulent transactions.

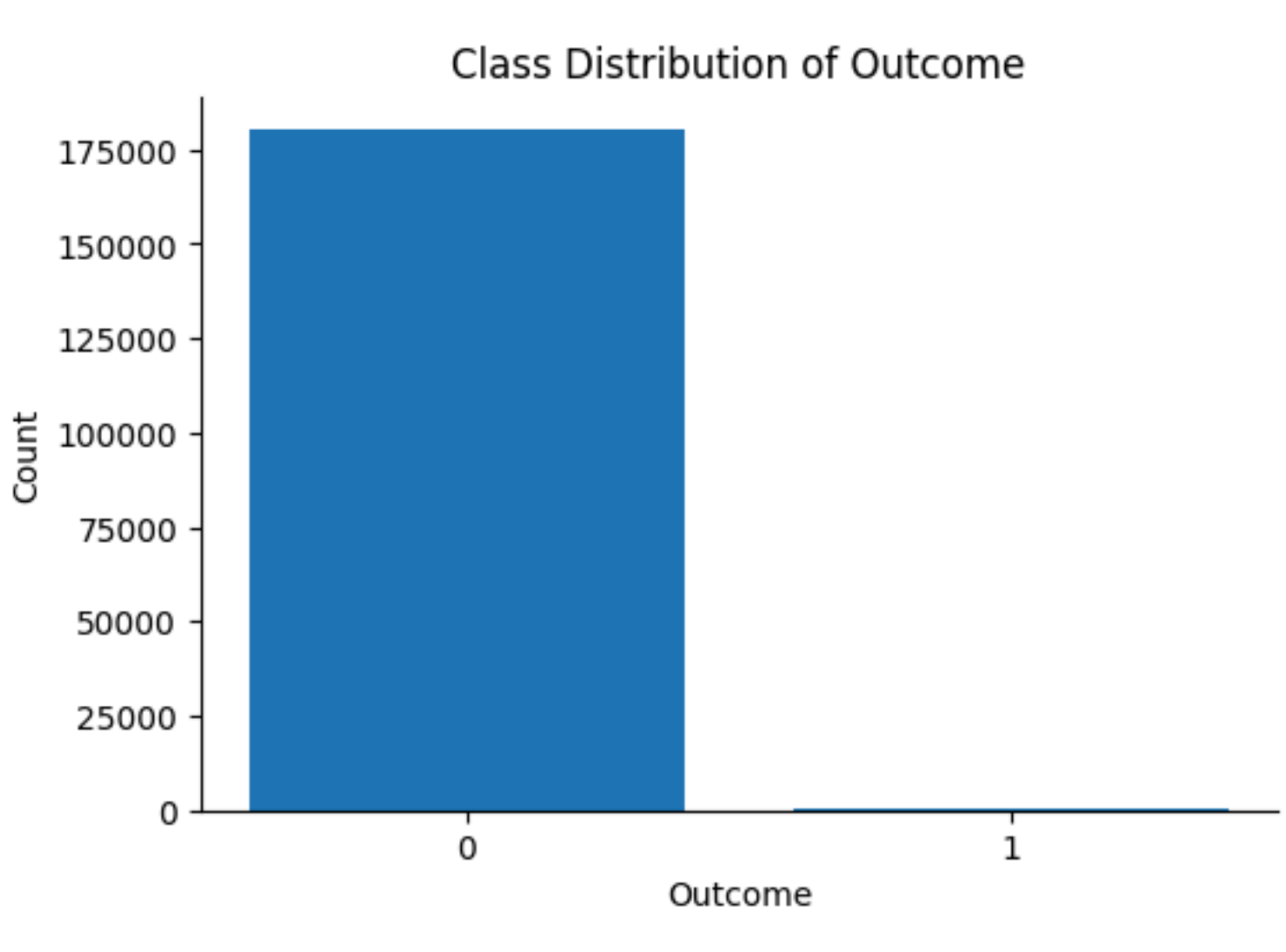

Figure 1. Class Distribution of Outcome

Second, traditional static rules-based fraud detection may not be effective when applied to this dataset. These rules rely on hard thresholds that flag any transaction that goes over a specified monetary limit (Vedešin, 2025). As shown in the box plot in Figure 2, the median transaction amount for fraud cases is actually lower than that of non-fraud cases. Furthermore, even the outliers among fraudulent transactions are not as extreme when compared to non-fraudulent ones. While non-fraud transactions can reach values on the order of 10^4, the highest fraudulent transactions only reach around 10^3. This shows that fraudsters may be aware of these rules or thresholds and purposefully keep their transactions low to avoid detection. To counter this there must be additional layers of fraud detection to catch these lower value fraud transactions.

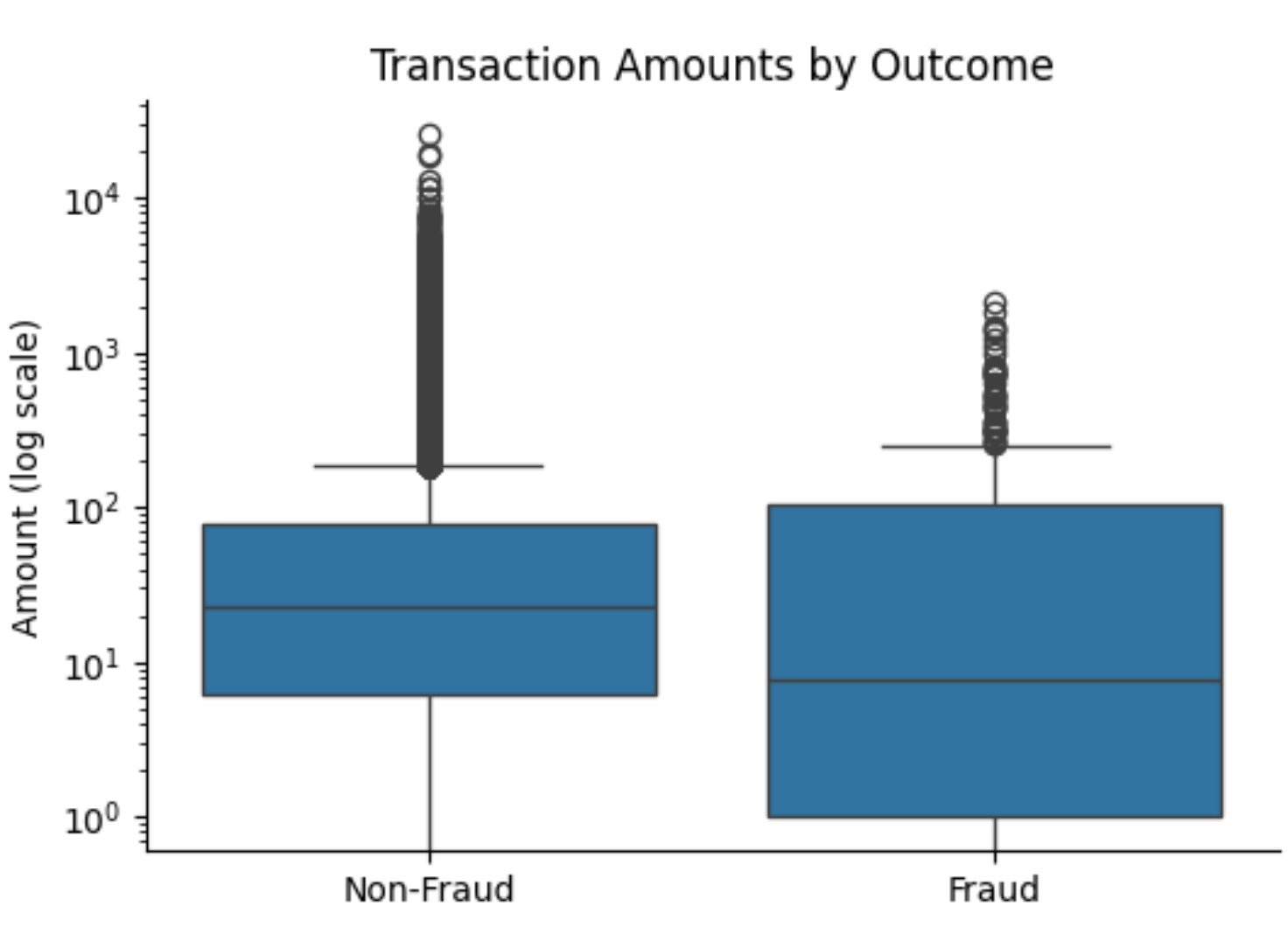

Figure 2. Box Plot of Transaction Amounts by Outcome

In contrast to using only transaction amounts, we can also look at the principal components. Figure 3 shows little to no correlation among non-fraudulent transactions while fraudulent transactions display stronger relationships in the heatmap. This suggests that fraud data contains patterns that machine learning can exploit to improve prediction.

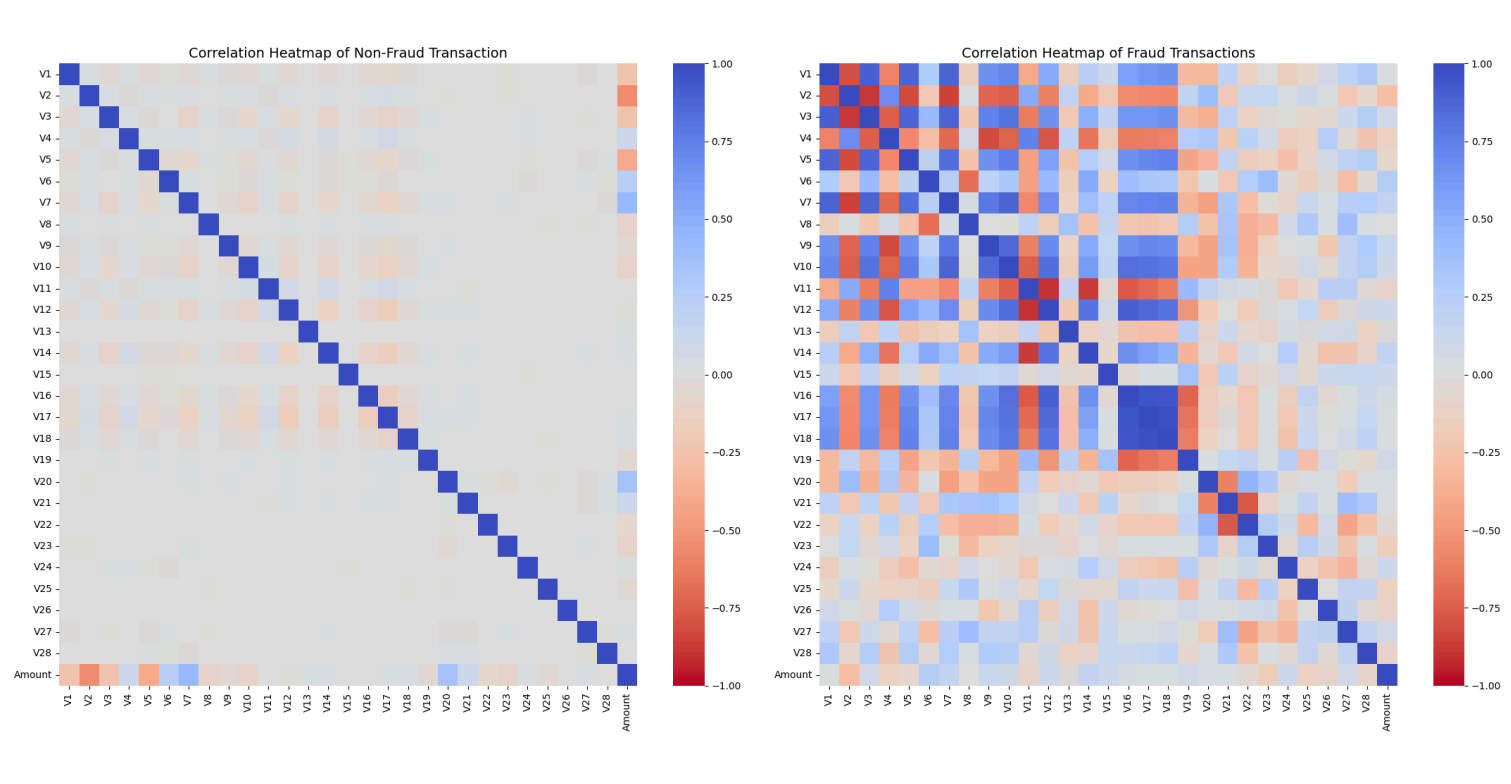

Figure 3 Correlation Heatmap of Non-Fraud and Fraud Transactions

To further visualize the patterns in fraud data, two scatterplots were made in Figure 4 to show the relationships of V1 to V3 (a) and V18 to V16 (b). These plots further emphasize how fraud transactions (red) cluster differently from non-fraud ones (blue), suggesting that certain components capture meaningful distinctions between the two classes.

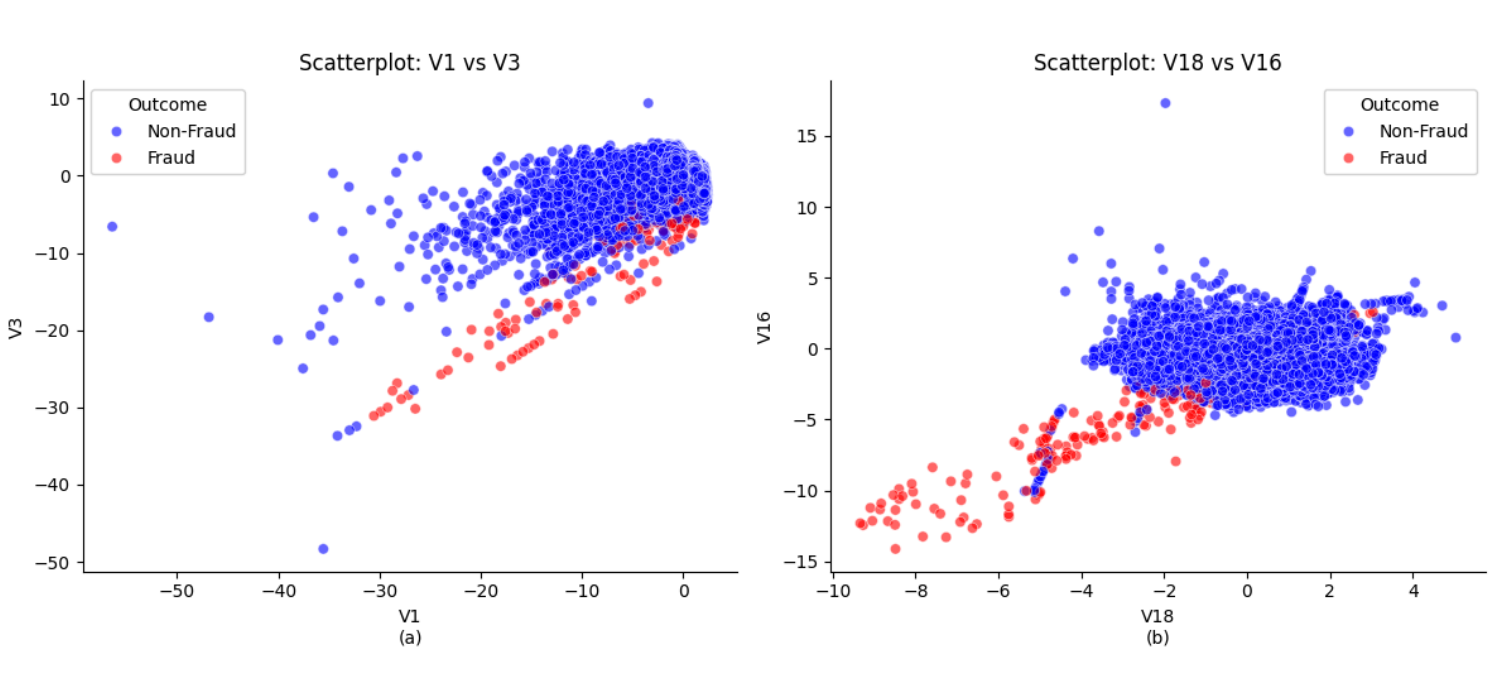

Figure 4 Scatterplots of Principal Components Highlighting Fraud vs. Non-Fraud Clusters

---

4.2 Handle Class Imbalance

4.2.1 Traditional Rebalancing Methods

From our exploratory data analysis, it is clear that most fraudulent transactions occur at smaller amounts, making simple thresholding ineffective. Moreover, the correlation matrices and scatterplots reveal distinguishable patterns that machine learning models can learn from.

Building on these insights, we now turn to model evaluation and introduce new key metrics to assess performance.

We first defined Detection Rate as Macro Recall, where it measures the model's overall ability to identify both fraudulent and legitimate transactions

$$\text{Detection Rate} = \frac{1}{2}\left(\frac{TP_{\text{fraud}}}{TP_{\text{fraud}} + FN_{\text{fraud}}} + \frac{TP_{\text{nonfraud}}}{TP_{\text{nonfraud}} + FN_{\text{nonfraud}}}\right)$$

Then, we defined Fraud Capture Rate, which reflects the proportion of fraud cases successfully captured

$$\text{Fraud Capture Rate} = \frac{TP_{\text{fraud}}}{TP_{\text{fraud}} + FN_{\text{fraud}}}$$

Finally, we have net value as the value of fraud captured (True Positive) minus the value of those transactions that were innocently flagged as fraud (False Positive).

$$\text{Net Value} = V_{\text{caught}} - V_{\text{innocently flagged}}$$

	Baseline	RUS	ROS	SMOTE	ADASYN	TOMEK-LINKS	Class Weights
(Macro) Precision	96%	53%	97%	94%	93%	96%	98%
(Macro) Recall	87%	93%	88%	91%	90%	88%	86%
(Macro) F1-Score	91%	54%	92%	92%	91%	91%	91%
Fraud Capture Rate	75%	89%	75%	81%	80%	75%	72%
Detection Rate	87%	93%	88%	91%	90%	88%	86%
Absolute Net Value Gained	-2079.44	-238,896	3464	3341	3316	-2076	3144.63
Relative Net Value Gained	-23%	-4,445%	64%	60%	59%	-23%	59%

Table 1. Metrics of Each Approach to Handling Class Imbalance

As seen in Table 1, Random Undersampling, highlighted in red, achieved the highest FCR (89%), but this came at the expense of significantly lower precision (53%) and F1-score (54%). In contrast, SMOTE, highlighted in green, offered a more balanced performance, with an 81% FCR while maintaining high precision (94%) and F1-score (92%). This balance shows that SMOTE not only detects a substantial portion of fraudulent transactions but also minimizes misclassification of legitimate ones, making it the more effective and reliable approach for CreditByte's fraud detection system. Thus, we recommend SMOTE as the approach for handling class imbalance. This claim is reinforced with the research by Cheah et al. where they showed SMOTE can provide relevant improvements on the detection of rare fraud cases by generating synthetic minority samples that improve classifier sensitivity (2023). However, we can build on SMOTE to see if there are hybrid strategies that can further improve our metrics.

4.2.2 Building on SMOTE

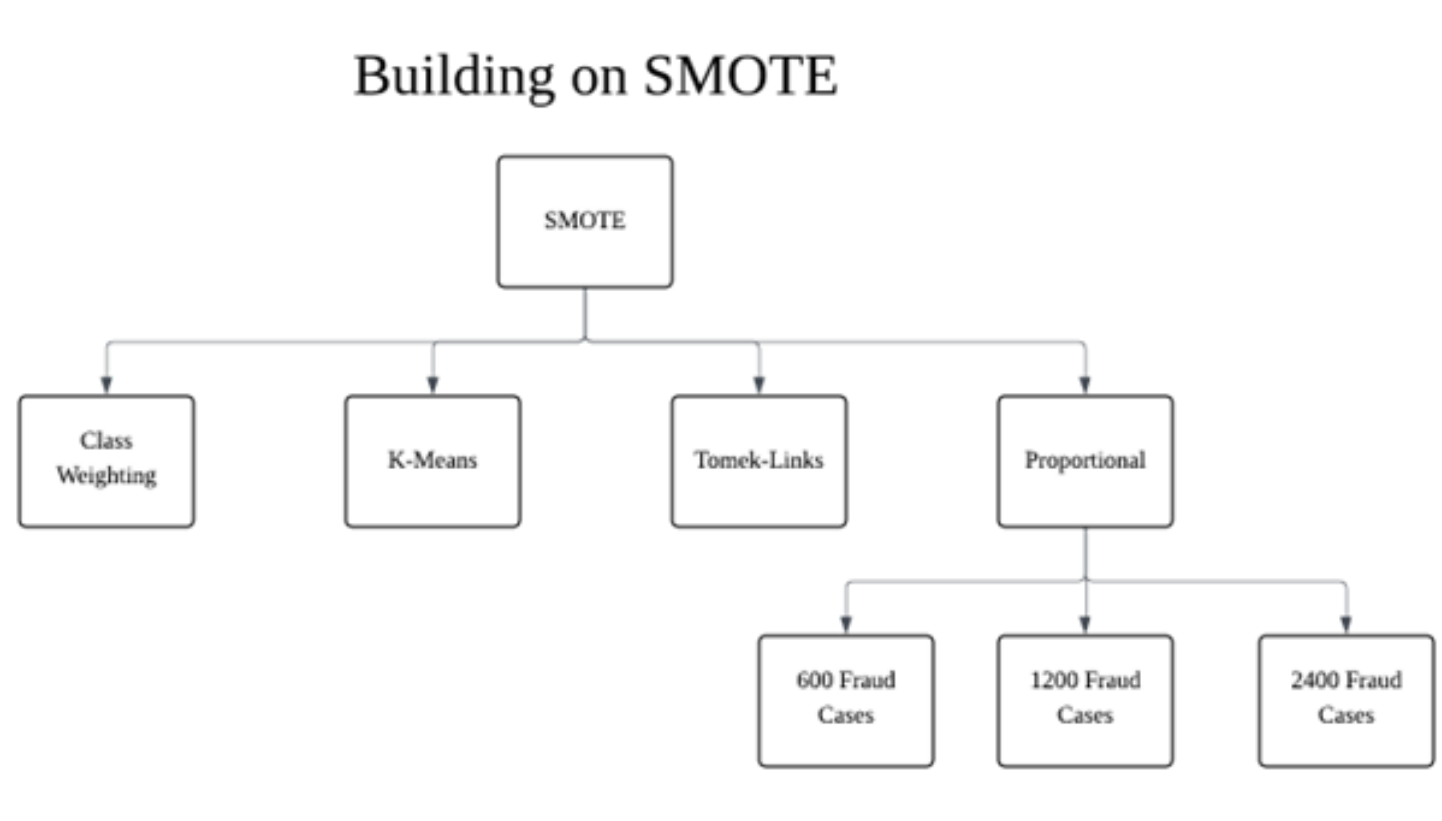

Figure 5. Building on SMOTE: Hybrid Oversampling Strategies

As seen in figure 5, to further optimize the SMOTE method, we used various hybrid strategies such as K-means, Tomek-Links, Class weighting, and Proportional SMOTE. K-means and SMOTE helps cluster the minority samples first before generating synthetic points. This ensures that the new samples are produced in more representative regions which avoids generating noisy points. Wang (2025) also used a similar approach demonstrating that combining SMOTE-KMeans with ensemble learning shows strong performance in fraud detection, further highlighting that rebalancing is a key step for handling skewed datasets. Tomek-Links and SMOTE removes borderline samples to clean overlapping instances which improves class separability. Additionally, the team implemented class weighting with SMOTE to give higher importance to the minority class during training through a weighted loss function.

All of these three hybrid strategies used typical libraries from `imblearn` but the team also created proportional SMOTE to define how many fraud cases there should be. According to a study, it found that balancing classes by making the minority approximately equal to the majority class is not always optimal (Wainer, 2016). It mentioned that moderate oversampling sometimes gives better results but if you oversample too much, you will get diminishing returns (Wainer, 2016). Using less-aggressive ratios would allow us to use the least amount of synthetic data while keeping the same recall and precision performance.

	Original SMOTE	SMOTE and K-means	SMOTE and Tomek-links	SMOTE and Class weighting	Prop. SMOTE (600 Fraud)	Prop. SMOTE (1200 Fraud)	Prop. SMOTE (2400 Fraud)
(Macro) Precision	94%	96%	94%	94%	93%	94%	93%
(Macro) Recall	91%	88%	91%	91%	91%	91%	91%
(Macro) F1-Score	92%	92%	92%	92%	92%	92%	92%
Detection Rate	81%	77%	81%	81%	82%	82%	81%
Fraud Capture Rate	91%	88%	91%	91%	91%	91%	91%
Absolute Net Value Gained	3341	3128.56	3323.96	3340.68	3336.38	3358.32	3271.40
Relative Net Value Gained	60%	57%	59%	60%	60%	60%	59%

Table 2. Metrics of SMOTE Hybrid Strategies

Based on the results of Table 2, it is notable that all of these hybrid SMOTE strategies differ marginally from the original SMOTE in terms of performance. Across various evaluation metrics, the variations are relatively small, which implies that the baseline SMOTE is already robust for this dataset. For instance, the F1-score remains consistently at 92% across all strategies which indicates that the overall balance between precision and recall is unaffected by the choice of hybrid method.

K-means and SMOTE improves precision from 94% to 96% but lowers recall to 88%. Tomek-Links and Class weighting with SMOTE perform almost the same as its baseline suggesting limited added value. Proportional SMOTE tested at different oversampling levels, shows small differences in net value ranging from approximately 3,300 to around 3360. While Proportional SMOTE with 1200 fraud cases shows a higher net value gained and detection rate than the baseline, the 4% standard deviation indicates that the improvement may not be consistently reliable. Overall, despite their theoretical advantages, these refinements offer minor improvements over standard SMOTE for this dataset.

4.3 Limitations

The results of this report are limited by the initial conditions of the machine learning model. Only a Random Forest with 500 estimators was used as a baseline, so no other models and hyperparameter tuning was used to further improve performance.

Due to how the data was first given, the features were anonymized and PCA-transformed before it met the hands of the authors, so feature engineering, and initial feature analysis was unavailable. Since the features are non-interpretable, explainability techniques like LIME and SHAP can't be effectively used to provide reasons why certain transactions are flagged. As mentioned before the sample size has a relatively low number of fraudulent transactions at 305 cases, which severely limits how the machine learning model can learn patterns. The best performing class balancing method SMOTE works by creating synthetic data, so there is a risk that performance may degrade when tested on completely new datasets.

---

5. Conclusion and Recommendations

In future studies, we would like to try rebalancing and explore alternative machine learning models, such as gradient boosting (e.g., XGBoost, LightGBM) or deep learning, to compare their performance with the Random Forest baseline. We could also test ensemble methods that combine multiple classifiers to better capture different aspects of fraud patterns. We should also test real-time detection by simulating how models perform in a streaming environment since fraud prevention often requires immediate decisions. We should also examine the long-term stability of models by testing performance across time windows to see how quickly fraud patterns change and whether retraining frequency affects reliability. And finally, we should expand the research to include explainability techniques, such as SHAP values or LIME, so that fraud detection systems remain interpretable and trustworthy for both technical teams and business stakeholders.

In conclusion, CreditByte's problem was class imbalance in the data, where legitimate transactions outnumbered fraudulent ones, making detection unreliable. Through the EDA, it was clear that fraudsters operate with smaller transaction amounts and can bypass conventional rules, while patterns in the data can be leveraged by machine learning. This shows that the traditional thresholding is not effective since the small fraud transactions do not get flagged. By testing different rebalancing strategies such as Random Undersampling, Random Oversampling, SMOTE, ADASYN, Tomek Links, and Class Weighting the data showed that these newer machine learning methods, specifically SMOTE, outperform simple thresholding. This shows that addressing the imbalance improves both the technical performance of the models and the business objective of reducing financial losses. In the end, the report shows that machine learning can strengthen fraud detection, giving CreditByte a more robust and adaptive defense against evolving fraud tactics.

References

Cheah, P. C. Y., Yang, Y., & Lee, B. G. (2023, September 5). Enhancing financial fraud detection through addressing class imbalance using Hybrid smote-gan techniques. MDPI. https://www.mdpi.com/2227-7072/11/3/110

Federal Trade Commission. (2025, March 10). New FTC Data Show a Big Jump in Reported Losses to Fraud to $12.5 Billion in 2024. U.S. Federal Trade Commission. https://www.ftc.gov/news-events/news/press-releases/2025/03/new-ftc-data-show-big-jump-reported-losses-fraud-125-billion-2024

Vedešin, A. (2025). What Are Fraud Detection Rules: Types and Best Practices. Vespia. https://vespia.io/blog/fraud-detection-rules

Wainer, J. (2016). An empirical evaluation of imbalanced data strategies from a practitioner's point of view (Technical Report 1606.00930). arXiv. Retrieved from https://arxiv.org/abs/1606.00930

Wang, Y. (2025, March 27). A data balancing and Ensemble Learning Approach for credit card fraud detection. arXiv.org. https://arxiv.org/abs/2503.21160

Supplementary

Resampling Strategies

Oversampling

- Duplicate or generate more minority samples.
- **Pros:** Keeps all majority data
- **Cons:** Risk of overfitting

Undersampling

- Drop some majority samples.
- **Pros:** Fast, reduces dataset size
- **Cons:** Lose information

Oversampling Techniques

SMOTE (Synthetic Minority Oversampling Technique)

- Create new synthetic samples by interpolating neighbors.
- Safer than just duplication.
- **Variants:**
  - **SMOTETomek** = SMOTE + cleaning noisy points
  - **SMOTEENN** = SMOTE + Edited Nearest Neighbors
  - **ADASYN** = Adaptive sampling (harder examples get more samples)

SMOTEN (Synthetic Minority Over-sampling Technique for Nominal features)

- **Use case:** When you have **categorical features** (nominal, not ordinal).
- **Problem:** Standard SMOTE interpolates between numeric features and cannot directly handle categories (e.g., *Red* vs *Blue*).
- **How it works:**
  - For categorical features → picks the most frequent category among k-nearest neighbors.
  - For numeric features → interpolates as in SMOTE.
- **Advantage:** Works for **mixed data types** (numeric + categorical).

SMOTE-NC (Synthetic Minority Over-sampling Technique for Nominal & Continuous features)

- **Use case:** Datasets with **both categorical and continuous features**.
- **Difference from SMOTEN:**
  - SMOTEN → categorical-only
  - SMOTE-NC → hybrid datasets (categorical + continuous)
- **How it works:**
  - Continuous features → interpolation
  - Categorical features → nearest neighbor's category is chosen (no interpolation)
- **Advantage:** Can oversample in **mixed datasets** (e.g., fraud detection with `age [numeric]` and `transaction type [categorical]`).

ADASYN (Adaptive Synthetic Sampling)

- **Full name:** ADAptive SYNthetic Sampling
- **Main idea:** Focuses on harder-to-learn cases near the decision boundary.
- **How it works:**
  - Identify minority samples with many majority neighbors → "hard" examples.
  - Generate more synthetic points around these hard examples.
  - Easier minority samples → fewer synthetic samples.
- **Advantage:** Creates a **more balanced decision boundary** → model learns rare but important cases.
- **Disadvantage:** May introduce noise if misclassified points are oversampled too much.

When to Use Each

- **SMOTEN** → categorical-only datasets
- **SMOTE-NC** → mixed categorical + continuous datasets
- **ADASYN** → highly diverse minority examples, focusing on difficult boundary cases

Details on Metrics

1. Detection Rate
   - This asks, "Out of all transactions, how well does the model is at correctly identifying both fraud and non-fraud overall, where both are equally important?"
2. Fraud Capture Rate
   - This asks, "Out of all the fraud cases, how many did the model actually catch?"
3. Net Value Gained -
   - This asks, "How much money does the model helps save (by catching fraud)?"
4. **Detection Rate which is the same as Macro Recall**
   Detection Rate:

$$\text{Detection Rate} = \frac{1}{2}\left(\frac{TP_{\text{fraud}}}{TP_{\text{fraud}} + FN_{\text{fraud}}} + \frac{TP_{\text{nonfraud}}}{TP_{\text{nonfraud}} + FN_{\text{nonfraud}}}\right)$$

where

- n_{fraud} = number of fraud samples
- n_{nonfraud} = number of non-fraud samples
- N = total number of samples
- TP_{fraud} = fraud cases correctly identified
- FN_{fraud} = fraud cases missed
- TP_{nonfraud} = non-fraud cases correctly identified
- FN_{nonfraud} = non-fraud cases missed

2. **Fraud Capture Rate, which is the same as Recall for Fraud Transactions**

$$\text{Fraud Capture Rate} = \frac{TP_{\text{fraud}}}{TP_{\text{fraud}} + FN_{\text{fraud}}}$$

where

- TP_{fraud} = fraud cases correctly identified
- FN_{fraud} = fraud cases missed

3. **Net Value**

$$\text{Net Value} = V_{\text{caught}} - V_{\text{innocently flagged}}$$

where

- V_{caught} = total value of fraud correctly detected
- $V_{\text{innocently flagged}}$ = total value of fraud incorrectly detected